UNIVERSITY OF PERADENIYA
Faculty of Engineering
Department of Electrical & Electronic Engineering

Design & Implementation of a Line Following Robot

using Discrete Logic Gates

EE2030 • DIGITAL LOGIC DESIGN & SYNTHESIS

GROUP 03

E/22/073	G.A.K.S. Devindya
E/22/076	D.A.S.R. Dharmarathna
E/22/077	E.T. Dheerasinghe
E/22/079	M.M.A. Dilhara
E/22/080	B.M.A.S. Dilshan
E/22/085	D.M.A.S. Dissanayake

Abstract

This report documents the design, iteration, and hardware implementation of an autonomous line-following robot built entirely from discrete logic integrated circuits. No microcontrollers, microprocessors, or programmable logic devices are used. The project was carried out for the **EE2030, Digital Logic Design and Synthesis** module during Semester 3 (January to May 2026) by a six-member team.

The final design (**V4**) employs three channels of a TCRT5000 infrared array feeding a two-stage combinational logic network of SN74HC00N NAND gates, SN74HC08N AND gates, and a SN74HC04N hex inverter, driven by a NE556N dual-timer PWM generator. Motor control is routed through the IN1 and IN3 inputs of an L298N dual H-bridge, enabling three per-motor states (FULL, PWM, and STOP) without any microcontroller in the loop.

The design passed through four architectural iterations: a flip-flop memory variant (V1), a bidirectional-drive variant (V2), an AND-only test build that ran the V4 circuit with PWM tied HIGH (V3), and the final PWM-driven design (V4). The team's research-first methodology, recorded in a continuous logbook, is what enabled these iterations to converge on a compact, competition-ready layout.

Contents

Abstract 1

1 Introduction 3
1.1 Problem Statement . . . 3

2 Project Objectives 3

3 Approaches Considered 3
3.1 Sensor Configuration . . . 4
3.2 Line-Loss Recovery . . . 4

4 Design Methodology 5

5 Design Evolution 5
5.1 V1, Truth-Table Logic with Flip-Flop Memory . . . 5
5.2 V2, Bidirectional Motor Control . . . 7
5.3 V3, AND Logic Validation with PWM tied HIGH . . . 7
5.4 V4, Full PWM with Brake-Mode Logic . . . 10
5.5 Version Comparison . . . 12

6 System Architecture (V4) 12
6.1 Signal-Flow Block Diagram . . . 12
6.2 Proteus Simulation . . . 12
6.3 IC Inventory and Gate Usage . . . 13
6.4 Logic Family Notes . . . 13
6.5 PWM Generation (NE556N) . . . 14
6.6 Power System . . . 15
6.7 Mechanical Layout . . . 16

7 Challenges and Solutions 16
7.1 Breadboard fault isolation . . . 16
7.2 Proteus simulation convergence . . . 16
7.3 TCRT5000 inverted polarity . . . 16
7.4 Left-drift on straight lines . . . 16
7.5 Sensor calibration fatigue . . . 17

8 Lessons Learned 17

9 Conclusion 17

References 17

A Bill of Materials 19

1 Introduction

The EE2030 module requires each group to build a line-following robot whose intelligence lives entirely in hardware. No firmware, no programmable logic devices, no shortcuts. Group 03 accepted this as a constraint worth leaning into, and spent the first four weeks of the semester on design rather than construction. The outcome is a robot whose control surface can be read directly off a single page of truth tables.

1.1 Problem Statement

Design and build a line-following robot that:

- Uses *only* discrete logic ICs. No MCUs, PLDs, or FPGAs.
- Tracks a black line of approximately **2.9 cm** width on a white surface, including dotted-line segments.
- The track includes curved sections. Some corners are roughly semicircular with a radius of about 21 cm, others are tighter with a radius of about 13 cm.
- Provides proportional steering rather than bang-bang ON/OFF response.
- Is competitive on speed and reliability for the EE2030 robot competition.

2 Project Objectives

1. **Primary.** Derive motor control signals from a 3-sensor IR array using combinational logic alone.
2. **Secondary.** Layer PWM speed modulation onto that logic for proportional steering.
3. **Tertiary.** Keep the IC and gate count as low as the truth-table allows.
4. **Documentation.** Maintain a continuous design logbook that captures every architectural decision and the reasoning behind it.

3 Approaches Considered

Three control philosophies were reviewed before the team committed to combinational logic with PWM.

Table 1: Line-following control strategies reviewed during planning

Approach	Summary	Verdict
Bang-bang (ON/OFF)	Sensor directly switches motor ON or OFF. Simple, but forces tight turns to reverse one wheel, which bleeds forward speed we cannot afford to lose.	**Rejected**
Op-amp proportional	Analog comparator chain driving the H-bridge. Avoids an MCU but adds analog calibration pain on top of digital logic.	**Rejected**
Multi-level discrete	Combinational logic chooses between FULL, PWM, and STOP per motor using a timer IC for duty cycle.	**Selected**

Why not bang-bang

Bang-bang steering forces one wheel to *reverse* for sharp corners. Reversing throws away the kinetic energy the robot needs to stay fast on the track, a direct speed penalty we could not accept for a competition build. Multi-level discrete control with PWM keeps both wheels always rotating forward and trades speed smoothly instead of stopping dead.

3.1 Sensor Configuration

The team weighed 2-, 3-, and 4-sensor layouts. 5+ sensor PID-style arrays were not considered. They presume analog or MCU processing that the project constraints forbid.

Table 2: Sensor-count trade study

Count	Advantages	Disadvantages
2 sensors	Minimum gate count	No centred state; constant oscillation
3 sensors	Centred detection, clean 8-state truth table	Balanced complexity
4 sensors	Finer error granularity	16-state truth table inflates gate count

Decision

Three sensors were chosen. The smallest sensor count that yields a distinct centred state, giving an 8-row truth table that maps to eight motor-pair states.

3.2 Line-Loss Recovery

Two recovery behaviours were evaluated: stateful flip-flop memory of the last detected side, and stateless momentum coast. The competition track has **dotted segments**. The robot must keep moving when all three sensors read white, so a momentum-based strategy is not merely simpler, it is required.

Decision

Momentum recovery. When all three sensors read white, both motors run at FULL so the robot coasts across dotted segments and re-acquires the line downstream. No sequential state is needed.

4 Design Methodology

Group 03 adopted a **research-first** methodology: derive, simulate, justify, then solder. The first several weeks of the semester produced no hardware at all, only a logbook. The planning phase covered:

1. Sensor configuration trade studies (2 vs. 3 vs. 4 sensors).
2. Truth-table derivation for every reachable sensor state.
3. Motor-control strategy comparison (ON/OFF, bidirectional, PWM).
4. Line-loss recovery analysis (flip-flop memory vs. momentum).
5. IC selection on the joint constraints of gate-count, propagation delay, and inter-family compatibility.
6. Chassis geometry (sensor lead distance, weight distribution).
7. Boolean reduction of the motor equations.

5 Design Evolution

The design matured through four architectural iterations. V1 and V2 were evaluated on paper only; V3 and V4 were physically realised. Each entry below links back to the primary logbook entry that captured the decision.

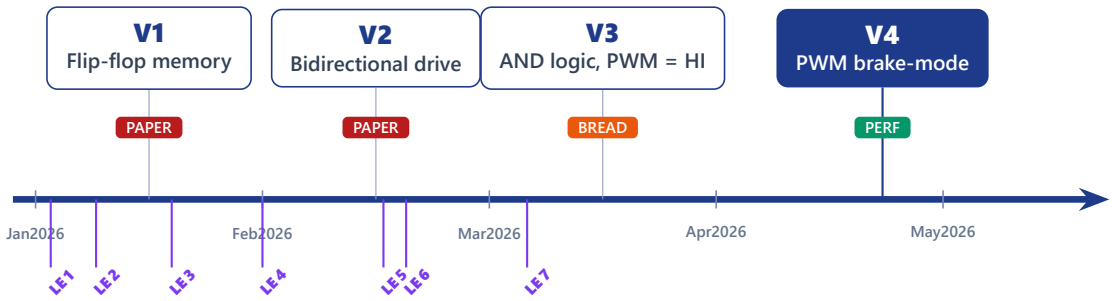


Figure 1: Design timeline. Top: the four version iterations with a PAPER / BREADboard / PERFboard build-state badge under each. Bottom: logbook entries that anchor each decision.

5.1 V1, Truth-Table Logic with Flip-Flop Memory

January 2026, paper design only

The first architecture used three IR sensors feeding a truth-table-driven combinational stage, with a `CD4013BE` D flip-flop holding the last known side of the line for recovery after line loss. A `74HC157` multiplexer selected between live sensors and memory output.

LOG ENTRY 1 • Jan 1, 2026 • Sensor selection & logic derivation

On the first working day of the semester, the team sat down to pick a sensor count and a control approach. Two-sensor and three-sensor configurations were compared, and three sensors won because only three gives a distinct "centred" state. The centre sensor was placed on the track midline with left and right sensors placed symmetrically. Truth tables

for left-motor and right-motor control were drafted, and the team committed to deriving motor control from plain logic gates rather than from a PID loop or microcontroller.

LOG ENTRY 2 • Jan 5, 2026 • Turning strategy & need for memory

The second meeting looked at how the robot should turn. The early sketch drove only the motor-positive rails and grounded the negatives, so the robot could only go forward or stop. For sharp corners that turned out to be too weak. The team split turns into "soft" (one motor on, one off) and "tight" (one motor forward, the other reversed) and debated how to recover when the robot fully leaves the line. Blind reversing was ruled out because it can push the robot further off-track; a memory element that remembers which side saw the line last was introduced instead. That pushed the design from pure combinational into sequential territory and pointed straight at flip-flops.

LOG ENTRY 3 • Jan 12, 2026 • D flip-flop recovery scheme finalised

The third session pinned down the recovery mechanism. A D-type flip-flop stores which side last detected the line. A small enable network ensures the flip-flop only updates when at least one sensor actually sees the line, so noise during line loss does not corrupt the stored side. A 2-to-1 multiplexer then arbitrates between live sensor data (when the line is visible) and the stored last-known-side (when every sensor reads white). The team drew the block diagram (sensors to MUX to D flip-flop to motor logic) and locked down the IC shopping list: `CD4013BE`, `74HC157`, `74HC00`, `74LS04`, `74LS08`, `NE556N`.

Planned truth table (motor enables, no PWM yet):

L	C	R	LM	RM	Action
0	0	0	Q	$\overline{Q}$	Recovery (last side)
0	0	1	1	0	Turn right
0	1	0	1	1	Forward
0	1	1	1	0	One-wheel right
1	0	0	0	1	Turn left
1	0	1	0	0	Stop (ambiguous)
1	1	0	0	1	One-wheel left
1	1	1	1	1	Forward

Table 3: V1 truth table. Q is the D-flip-flop output holding the last side that saw the line.

Not built

V1 was never actually built. When the team sat back and counted the cost, it needed **six ICs** (flip-flop, multiplexer, two logic families, an inverter, and the timer) and all the extra pitfalls that come with sequential logic: the flip-flop can latch the wrong side if it updates at the wrong moment, and the MUX has to be timed carefully to avoid glitching. On top of that, the competition track has dotted segments where every sensor reads white for a short stretch. Momentum coasting handles that case naturally, so storing "which side was I on last time" adds complication without adding value. V1 was retired before the first solder joint.

5.2 V2, Bidirectional Motor Control

February 2026, paper design only

Dropping the flip-flop gave a purely combinational design, but tight corners would still overshoot at speed. The V2 proposal reversed one wheel for sharp turns using all four L298N inputs `IN1` to `IN4`.

LOG ENTRY 4 • Jan 21, 2026 • Pre-lab two-sensor analysis

This was a class-assigned pre-lab exercise, not the main project work. The task described a five-state two-sensor line follower, and the team was asked to minimise the Boolean expressions for the four motor-driver inputs (IN1 through IN4), draw the state diagram, and propose an SR-latch implementation for the state memory. The team treated it as preparation for the next lab session. The exercise reinforced how quickly bidirectional control inflates the state space and pushed the group back toward a combinational-only approach for the actual project.

LOG ENTRY 5 • Feb 17, 2026 • Bidirectional truth table, then abandonment

The team mapped every sensor pattern to an explicit four-bit motor-driver output, allowing "tight" turns that reversed one wheel and "one-wheel" turns that stopped the other. On paper the logic was correct and produced the desired behaviour in every sensor state. In discussion however the failure modes stacked up fast: reversing one wheel causes mechanical wobbling on the chassis, rapid state switching during curves produces oscillation, and hard on/off transitions send jolts through the drivetrain. The group concluded that smoothness mattered more than sharpness, and switched the entire control philosophy to PWM-modulated proportional steering instead of bidirectional reversal.

L	C	R	IN1	IN2	IN3	IN4	Action
0	0	0	1	0	1	0	Forward
0	0	1	1	0	0	1	Tight right
0	1	0	1	0	1	0	Forward
0	1	1	1	0	0	0	One-wheel right
1	0	0	0	1	1	0	Tight left
1	0	1	0	0	0	0	Stop
1	1	0	0	0	1	0	One-wheel left
1	1	1	1	0	1	0	Forward

Table 4: V2 truth table, controlling `IN1` through `IN4` directly. "One-wheel" turns stop one motor and drive the other at full speed; PWM is not yet in the picture.

Not built

V2 was abandoned on paper. Reversing a wheel means kinetic energy is burnt as heat rather than used to corner, and the robot loses straight-line speed for no gain. The team committed to **proportional PWM steering** as the replacement path.

5.3 V3, AND Logic Validation with PWM tied HIGH

February to March 2026, breadboard then perfboard

V3 was not a separate architecture. It was a pragmatic test build of the V4 circuit with the PWM line tied to logic HIGH. Tying PWM high collapses every PWM cell in the V4 truth table into FULL, reducing the controller to three useful states: straight, left, right. This let the team validate the AND, NAND, and NOT wiring without yet trusting the timer stage.

V3a: The breadboard attempt

The first V3 build went on a breadboard in the lab.

LOG ENTRY 6 • Feb 19, 2026 • Breadboard build, partial success

The team assembled the full V3 circuit on a breadboard during the lab session: 7805 regulator with decoupling, the SN74LS04/08 logic stack, the NE556 timer, the L298N driver, and three IR sensors. Verification was done stage by stage. The 5 V rail read stable on a multimeter. Both motors spun freely when the board was powered. When sensors were waved over black and white strips however, the motors did not change speed or direction at all. The team tried to probe individual signals, but the breadboard had already accumulated hundreds of jumper wires and the density made it impossible to trace the fault. The fault could have been in the PWM generator, the logic stack, the sensor polarity, or the L298N wiring; there was no clean way to isolate it on that hardware. The team called the session a "needle in a haystack" and decided to rebuild on perfboard with structured signal routing before trying again.

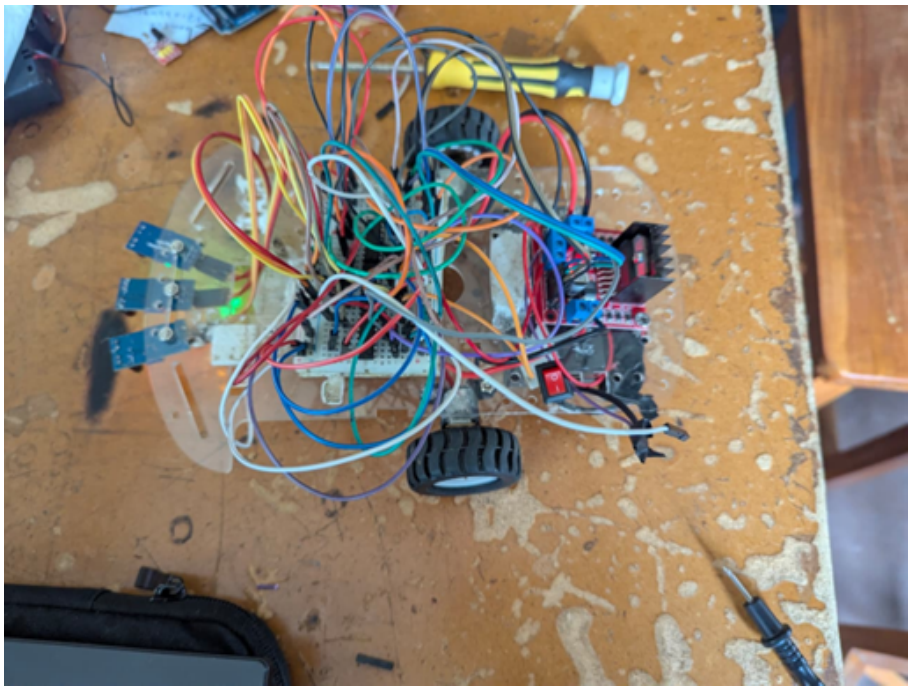


Figure 2: Breadboard V3 build, overview. The wire density that defeated debugging is visible.

Breadboard abandoned

The breadboard showed stable power rails, continuous motor rotation, and zero response to sensor input. Jumper density made fault localisation impossible. The team migrated to a **perfboard with soldered connections** before continuing.

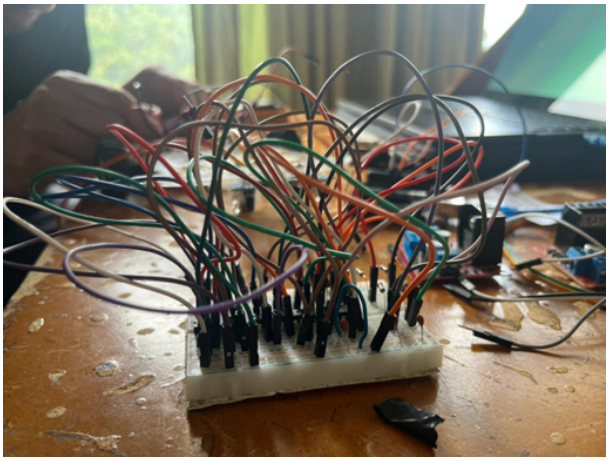


Figure 3: Logic-stage detail. The SN74LS04 and SN74LS08 were laid out side by side but shared rails complicated probing.

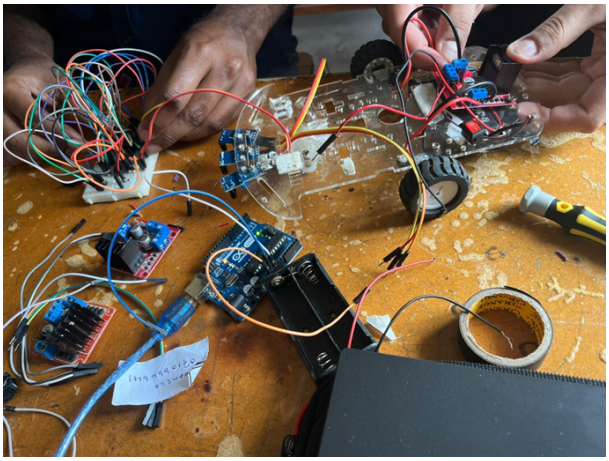


Figure 4: Motor and sensor wiring. The L298N module sits at the right; the three IR sensors feed in along the top rail.

V3b: Perfboard with Flying-Fish sensors

After the breadboard failure, the team rebuilt the circuit on perfboard with tidy signal routing and DIP sockets. To isolate whether the logic itself was sound, both PWM pins were tied to 5 V (logic HIGH). On this setup, the robot worked.

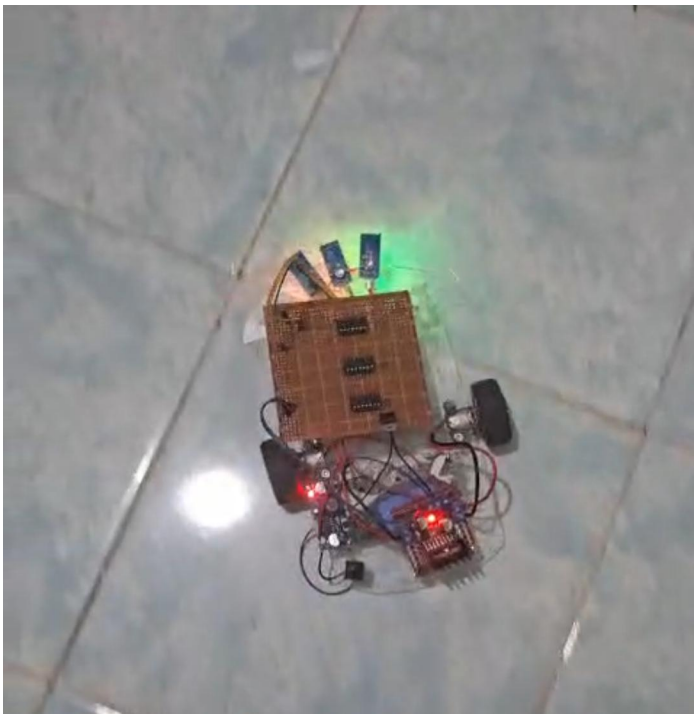


Figure 5: V3b perfboard build with three Flying-Fish IR modules.

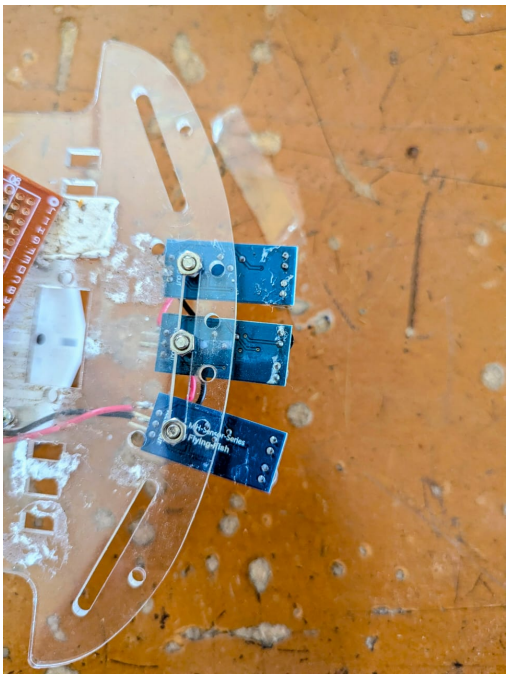


Figure 6: MH-Sensor Flying-Fish close-up. The threshold trimpot is on the *underside*.

Collapsed truth table (V3 with PWM

With PWM held HIGH every PWM cell in the V4 table becomes FULL. Only three distinct behaviours remain:

- Only C detects, straight.
- Only L detects, stop left motor, right motor runs.
- Only R detects, stop right motor, left motor runs.

Every other sensor pattern defaults to straight (both motors FULL).

With the Flying-Fish modules and PWM tied HIGH the robot **completed a full lap of the competition track in approximately 45 seconds**. A video of this run exists; the link is tracked in the project TODO file. One weakness remained: at one point on the track the printed black line had faded from many previous test runs, and the robot briefly lost the line there. The fade was not a circuit problem but a track condition.

Flying-Fish calibration pain

Each Flying-Fish module has a trimpot on its *underside* for black/white threshold adjustment. Every track repaint or lighting change required retuning all three trimpots, and because the trimpot faces downwards the robot had to be lifted off the track, tuned, and placed back on for every adjustment. For a competition build, this was unworkable.

V3c: Migration to TCRT5000

To fix the calibration problem the Flying-Fish modules were replaced with a front-mounted `TCRT5000` five-channel array. The robot uses only the middle three channels of that array; the outermost two channels are left unconnected.

TCRT5000 polarity is inverted

The combinational logic was derived assuming black surface outputs logic **1** (Flying-Fish convention). The TCRT5000 is opposite: it outputs **1** on white and **0** on black. Rewriting the entire logic to match would have meant rebuilding most of the perfboard. Instead, the team used three of the four unused gates in the `SN74HC04N` hex inverter to flip L, C, and R at the sensor inputs, so the NAND and AND stages still see “black = 1” exactly as they were designed.

5.4 V4, Full PWM with Brake-Mode Logic

March 2026 onward, perfboard, current design

V4 is the final architecture. PWM_A and PWM_B replace the logic-HIGH test wire, restoring the full three-level motor response (FULL, PWM, STOP). An `SN74HC04N` hex inverter does two jobs: it produces the outer complement in each motor expression (2 gates) and it flips the TCRT5000 polarity back to “black = 1” for the logic stages (3 gates), for a total of 5 inverter gates used. An `LM2596` buck converter was installed to give coarse motor-voltage trimming independent of PWM duty cycle.

LOG ENTRY 7 • Mar 5, 2026 • PWM & LM2596 added

Two additions went in together. First, an LM2596 DC-DC step-down module was inserted between the battery and the L298N motor rail so the overall motor voltage could be trimmed with a screwdriver instead of being locked to the battery voltage. Second, 50 kΩ multi-turn potentiometers replaced the fixed resistors on the NE556 timing network, giving the team fine control over PWM duty cycle per motor. Together these two changes turned the robot from an on-off device into one with two independent analog knobs: overall motor speed and per-motor PWM trim. The PWM truth table was updated to distinguish FULL, PWM, and BRAKE/STOP states on each motor.

After swapping Flying-Fish for TCRT5000 and enabling PWM, the robot **completed the same track in approximately 31 seconds**, a speed improvement of about 30% over V3b. The faded-line spot still tripped the robot, which reinforced that the fade is a track issue rather than a sensor issue. Two follow-up actions came out of this run. The sensor array needs to be lowered toward the ground for stronger contrast. PWM was tuned only by observation on that day because no multimeter was available, so the duty cycle is not yet at its optimal setpoint. A further attempt to match the robot's turning radius to the track corner radii (21 cm and 13 cm) showed that the geared N20 motors are old enough that their speed characteristics are no longer reliably measurable; future tuning will use multimeter duty-cycle readings plus slow-motion video to find the sweet spot.

Motor logic equations

The two H-bridge direction inputs that matter are IN1 (left motor) and IN3 (right motor). The complementary inputs IN2 and IN4 are tied to GND; ENA and ENB are tied permanently HIGH so the bridge is always enabled.

$$\mathrm{IN1}_{\mathrm{left}} = \overline{L \cdot \overline{C \cdot \mathrm{PWM}_A} \cdot \overline{R \cdot \mathrm{PWM}_A}} \tag{1}$$

$$\mathrm{IN3}_{\mathrm{right}} = \overline{R \cdot \overline{C \cdot \mathrm{PWM}_B} \cdot \overline{L \cdot \mathrm{PWM}_B}} \tag{2}$$

Each inner $\overline{X \cdot Y}$ is a single SN74HC00N NAND. The outer three-way AND is implemented in SN74HC08N. The final complement is a single SN74HC04N inverter per motor.

L298N polarity used

With IN2=IN4=GND, the bridge outputs follow the rule: IN1=**1** drives the left motor forward, and IN1=**0** stops it. A PWM on IN1 therefore produces a proportional forward speed equal to the PWM duty cycle. This is the mapping used for every motor state below.

Complete V4 truth table

L	C	R	IN1 (Left)	IN3 (Right)	Action
0	0	0	FULL	FULL	Line lost, momentum coast
0	0	1	FULL	STOP	Hard RIGHT turn
0	1	0	FULL	FULL	Centred, max speed
0	1	1	FULL	PWM	Gentle RIGHT
1	0	0	STOP	FULL	Hard LEFT turn
1	0	1	PWM	PWM	Slow forward (crossover)
1	1	0	PWM	FULL	Gentle LEFT
1	1	1	PWM	PWM	Wide-line slow forward

Table 5: V4 truth table. **1 = BLACK** detected after the input inverters, **0 = WHITE**. Highlighted row (1,0,1) is the crossover state that V3 mishandled.

5.5 Version Comparison

Table 6: Summary comparison across iterations

Feature	V1	V2	V3	V4
ICs	6 (logic + FF + MUX)	4	4	**4 all-HC**
Gate count	14+	8+	10	**13**
Sequential	Yes (D-FF)	No	No	No
PWM	No	No	Tied HIGH	**Yes**
Control pins	LM / RM enables	`IN1–IN4`	`IN1/IN3`	`IN1/IN3`
Recovery	Stored direction	—	Always straight	Momentum
Status	Paper only	Paper only	Breadboard fail, then PERF works	**Perfboard**

6 System Architecture (V4)

6.1 Signal-Flow Block Diagram

The V4 system groups into five functional blocks. The TCRT5000 array presents 5 IR channels, but only the middle three are wired into the logic. Sensor signals pass through a polarity-flip inverter before entering the NAND and AND network. A timer IC feeds PWM into the same network, and the logic output drives the motor bridge.

6.2 Proteus Simulation

Before any soldering, the V4 circuit was validated in Proteus 8. The simulation uses two separate NE555 timers in place of the NE556 (see Section 7.2 for why), but is otherwise a one-to-one representation of the perfboard wiring.

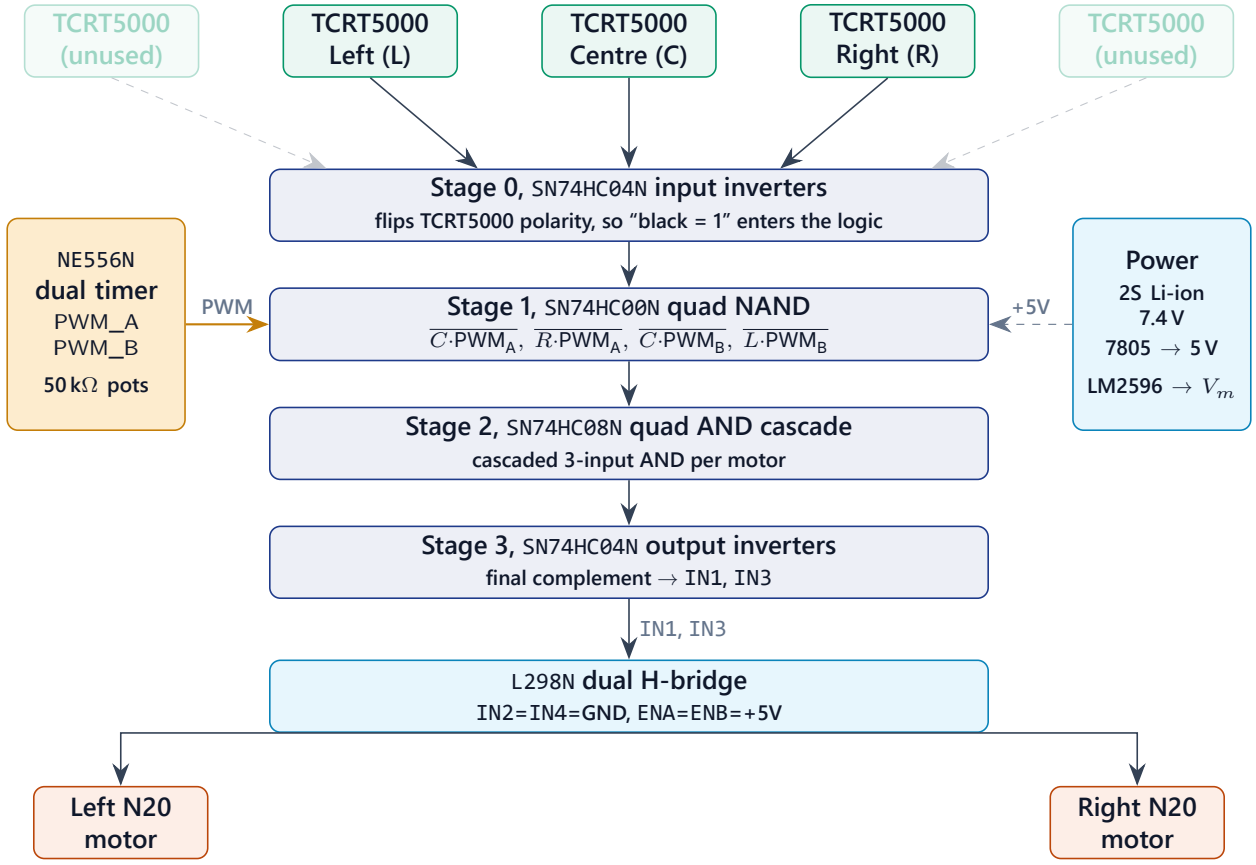


Figure 7: V4 signal-flow block diagram. Green is sensing (faded channels are present on the TCRT5000 board but unconnected); indigo is combinational logic; amber is PWM generation; teal is power and drive; orange is actuators.

6.3 IC Inventory and Gate Usage

Table 7: V4 logic-IC inventory (all 74HC family for consistent timing)

Ref	Chip	Function	Gates used	Role
U2	SN74HC00N	Quad NAND	4 of 4	Sensor · PWM inner products
U5	SN74HC08N	Quad AND	4 of 4	Cascade to form 3-way AND per motor
U6	SN74HC04N	Hex inverter	5 of 6	3 for TCRT5000 polarity, 2 for final complement
U3	NE556N	Dual timer	—	PWM_A, PWM_B generation

Gate budget: 4 NAND + 4 AND + 5 NOT = **13 gates across 3 logic ICs**, plus the NE556 timer.

6.4 Logic Family Notes

All logic ICs are **74HC CMOS family**, chosen over the mixed LS/HC combination considered in earlier drafts. Keeping to a single family:

- removes the V_{OH}/V_{IH} and V_{OL}/V_{IL} compatibility analysis needed when CMOS drives TTL;
- gives consistent propagation delays ($t_{pd} \approx 9$ ns at 5 V) across the whole pipeline;
- lets the PWM waveform pass through the logic without level loss.

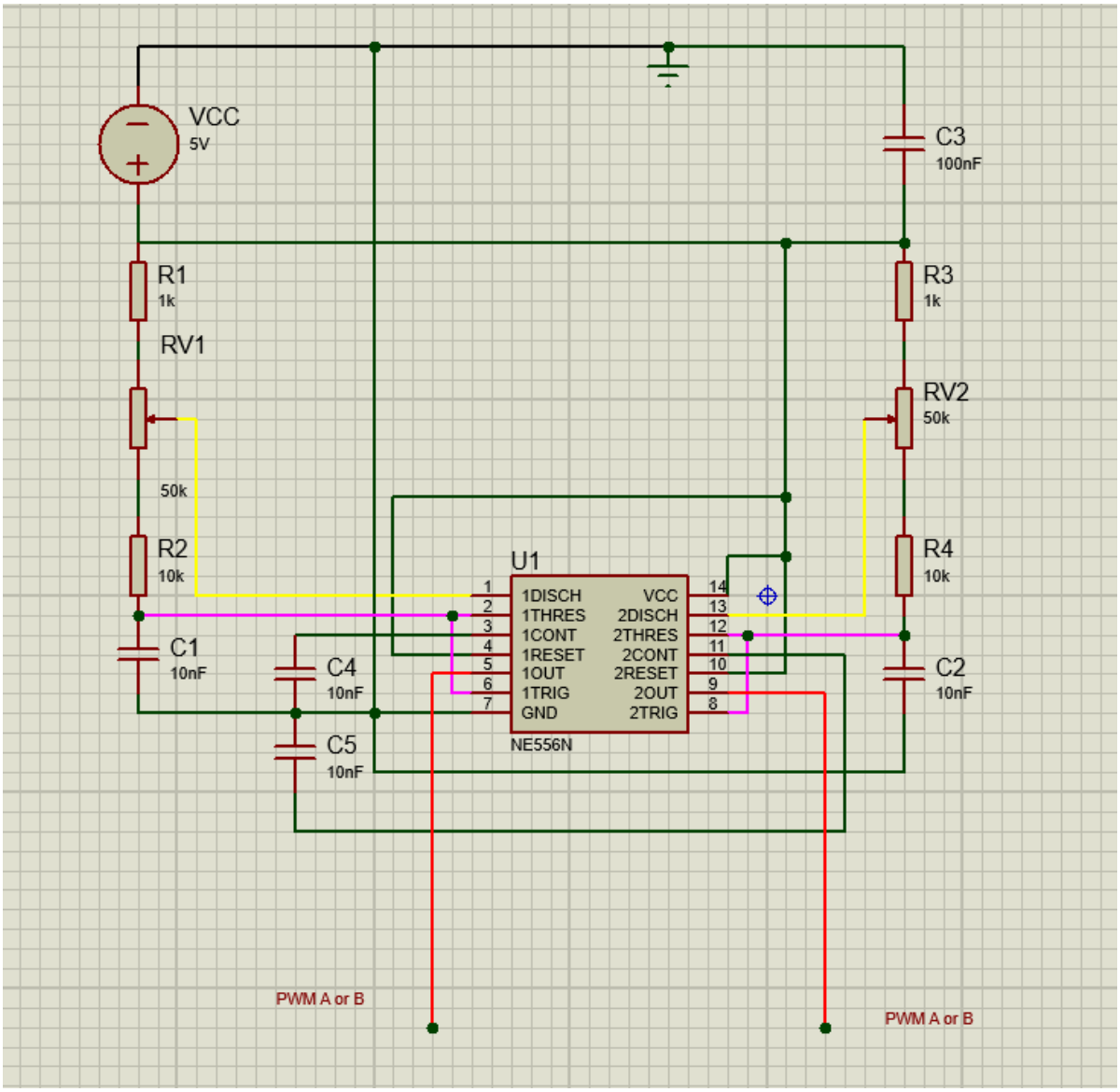


Figure 8: NE556N timing stage, Proteus wiring diagram. Each half of the NE556 runs as an independent astable with its own 50 kΩ trimpot on R_A.

6.5 PWM Generation (NE556N)

The NE556N runs both timers in astable mode with a safety resistor, a 50 kΩ trimpot on R_A, and fixed R_B:

$$f = \frac{1.44}{(R_A + 2R_B)\,C}, \qquad D = \frac{R_A + R_B}{R_A + 2R_B} \times 100\% \tag{3}$$

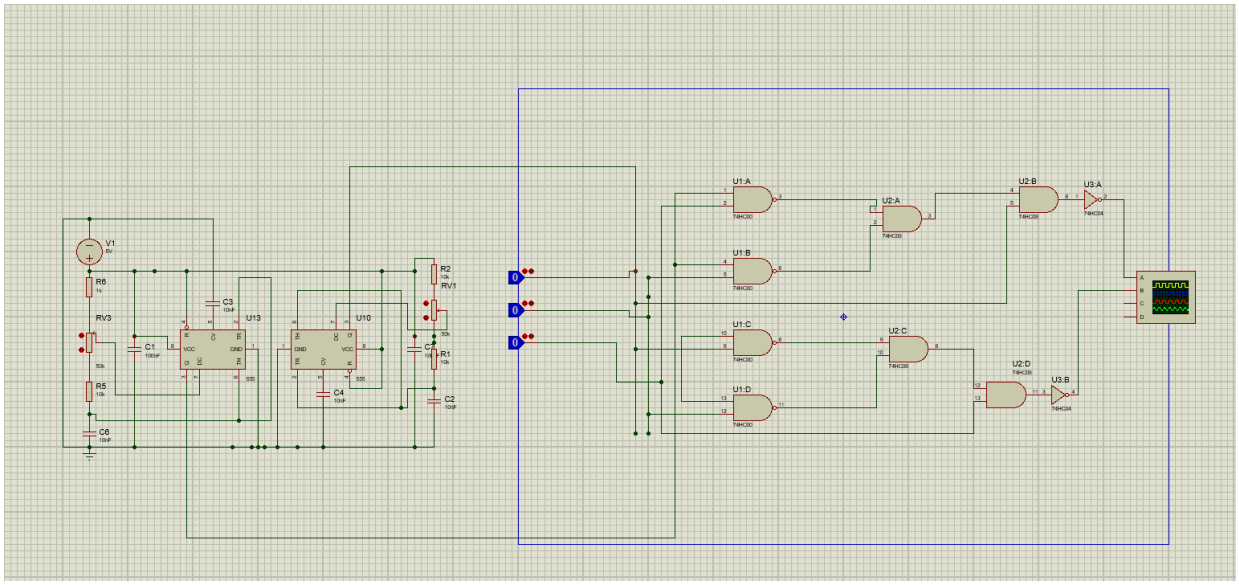


Figure 9: Complete robot Proteus simulation. Three sensor inputs feed the polarity inverters, the NAND and AND cascade, and the final output inverters into an L298N model driving two DC motors.

Table 8: PWM operating points with the 50 kΩ trimpot (R_B=10 kΩ, C=10 nF, R_{safety}=1 kΩ)

Trimpot	Duty cycle	Frequency
0 ($R_A = 1\,\text{k}\Omega$)	52.4%	∼6.9 kHz
1.2 kΩ	∼55%	∼5.7 kHz
5 kΩ	64%	3.8 kHz
20 kΩ	76%	2.0 kHz
50 kΩ	85.9%	∼1.1 kHz

Independent trimpots per timer let the team tune each motor's PWM separately to compensate for the 5 to 15% speed scatter normal to N20 gear motors.

6.6 Power System



Figure 10: Power distribution. Two independent rails: a 7805-regulated 5 V logic rail, and an LM2596-adjustable motor rail.

Known power constraints

- The 7805 drops out below ∼7 V input; the 2S pack must stay above 7.5 V under motor load.
- Heavy motor acceleration pulls the pack momentarily below that threshold; mitigated by a 100 μF reservoir on the L298N V_s terminal.

6.7 Mechanical Layout

The chassis uses a "hammerhead" geometry. Sensors extended 80 to 120 mm ahead of the drive axle, so small angular deviations translate into large lateral sensor displacement, giving earlier error feedback.

Table 9: Chassis dimensions

Parameter	Value
Overall size	∼180 × 150 mm
Sensor lead distance	80 to 120 mm ahead of drive axle
Sensor spacing	∼2.2 cm between centre and each side sensor
Sensor height	∼1.8 cm above ground
Wheel diameter	40 mm
Wheel track width	14 cm

The competition track has curved sections of two distinct radii, approximately 13 cm and 21 cm, which the robot must negotiate without slowing appreciably.

7 Challenges and Solutions

7.1 Breadboard fault isolation

The V3 breadboard (hundreds of jumpers) gave stable 5 V, continuous motor motion, and no response to sensor stimulus. The cause was signal-routing density, which prevented localising the fault between PWM, logic, polarity, and driver. **Resolution:** migrate to perfboard with DIP sockets, separate power, logic, and motor zones, and labelled net traces.

7.2 Proteus simulation convergence

The NE556 SPICE model in Proteus 8 reports "Gmin step failed" and "Too many iterations without convergence" errors when both of its internal timers share the ground node. **Resolution:** replace the NE556 with two NE555 instances in simulation. The physical build uses the NE556 without issue.

7.3 TCRT5000 inverted polarity

The combinational logic was derived under the Flying-Fish convention where a black surface outputs logic 1. When the team later switched to the TCRT5000 sensor array, the array reported the opposite mapping: white outputs 1, black outputs 0. Rewriting all of the Boolean equations would have meant rebuilding most of the perfboard. **Resolution:** three of the four unused gates in the SN74HC04N hex inverter were pressed into service to flip L, C, and R at the sensor inputs. Downstream logic continues to operate in the "black = 1" frame exactly as originally designed, and the fix added zero new ICs.

7.4 Left-drift on straight lines

With the two N20 gear motors run from a single shared duty cycle the robot consistently drifts left on nominally straight segments. The left motor is measurably slower than the right on the current pair of motors. **Resolution:** two independent trimpots (one per NE556 timer) give per-motor PWM calibration so the left motor can be pushed at a slightly higher duty cycle than the right until the straight-line drift is nulled.

7.5 Sensor calibration fatigue

Before the TCRT5000 migration, each Flying-Fish module needed trimpot retuning every time the track was repainted or lighting changed. The Flying-Fish trimpot faces downward, so the robot had to be lifted off the track for every adjustment. **Resolution:** swapped to a `TCRT5000` front array with a single fixed comparator threshold per channel. No more per-run retuning.

8 Lessons Learned

Plan before building. Four weeks of paper design saved more than that in avoided hardware rework. V1's flip-flop recovery would have contradicted the dotted-line momentum strategy; V2's bidirectional drive would have cost straight-line speed.

Breadboards scale badly. Past ∼30 connections, permanent solder on perfboard is not optional for a circuit that has to be debugged.

Brake vs. coast matters. Active-drive H-bridge control (polarity via `IN1` and `IN3`, with `ENA` and `ENB` permanently high) gives sharper corner correction than coasting via the enable lines.

Identical motors do not exist. Independent per-motor PWM calibration is required. A single shared duty cycle will always drift.

Simulation models lie sometimes. The NE556 SPICE model in Proteus 8 has a known convergence bug. Physical hardware test disagrees with the simulator, and hardware wins.

Sensor lead distance is leverage. Extending sensors 80 to 120 mm ahead of the drive axle trades braking distance for earlier error feedback, and the earlier feedback wins at competition speeds.

9 Conclusion

Group 03's line-following robot achieves autonomous black-line tracking using **4 ICs, 13 logic gates, and no firmware**. The V4 architecture, an `SN74HC00N` NAND stage, `SN74HC08N` AND cascade, `SN74HC04N` inverter used for both TCRT5000 polarity correction and final output complement, and an `NE556N` PWM generator feeding an L298N bridge, fits cleanly on a single perfboard.

This report has documented the four architectural iterations that led here: a flip-flop-memory V1 that collided with dotted-line recovery requirements; a bidirectional-drive V2 that traded speed for sharpness; a V3 AND-only test circuit that first failed on a breadboard and then worked on perfboard with PWM tied HIGH (approximately 45 s lap time with Flying-Fish sensors); and the final V4 that restores PWM and pairs the logic with a TCRT5000 front array (approximately 31 s lap time). The logbook, preserved verbatim in callouts throughout this document, is the primary evidence that the design decisions were reasoned rather than improvised.

References

[1] Texas Instruments, "SN74HC00 Quadruple 2-Input Positive-NAND Gates," Datasheet, 2024.

[2] Texas Instruments, “SN74HC08 Quadruple 2-Input Positive-AND Gates,” Datasheet, 2024.

[3] Texas Instruments, “SN74HC04 Hex Inverter,” Datasheet, 2024.

[4] Texas Instruments, “NE556 Dual Precision Timer,” Datasheet, 2024.

[5] STMicroelectronics, “L298 Dual Full-Bridge Driver,” Datasheet, 2024.

[6] Texas Instruments, “μA7805 Fixed Positive Voltage Regulator,” Datasheet, 2024.

[7] Texas Instruments, “LM2596 SIMPLE SWITCHER Power Converter,” Datasheet, 2024.

[8] Vishay, “TCRT5000 Reflective Optical Sensor,” Datasheet, 2023.

[9] M. Pakdaman and M. Sanaatiyan, “Design and Implementation of Line Follower Robot,” *Int. Conf. Computer & Electrical Engineering*, 2009.

A Bill of Materials

Table 10: Complete V4 bill of materials

Component	Specification	Qty
Integrated Circuits		
SN74HC00N	Quad NAND gate (DIP-14)	1
SN74HC08N	Quad AND gate (DIP-14)	1
SN74HC04N	Hex inverter (DIP-14)	1
NE556N	Dual timer (DIP-14)	1
7805	5 V linear regulator	1
DIP-14 sockets	IC sockets	4
Modules		
L298N module	Dual H-bridge motor driver	1
LM2596 module	Adjustable DC-DC step-down	1
TCRT5000 module	5-channel reflective optical array, 3 used	1
Passives		
1 kΩ resistor	Trimpot safety	2
10 kΩ resistor	NE556 R_B	2
50 kΩ 3296W trimpot	Adjustable PWM duty	2
100 nF (104) ceramic	IC decoupling, 7805 bypass	5
10 μF electrolytic	7805 output filter	1
100 μF electrolytic	L298N motor-rail reservoir	1
10 nF (103) ceramic	NE556 timing + CTRL bypass	4
Mechanical		
N20 gear motor	Drive motors	2
40 mm wheels	Drive wheels	2
Caster wheel	Front support	1
18650 Li-ion cell	3.7 V, configured 2S	2
Perfboard	Circuit mounting	1
Chassis	Robot frame	1